ClinicHub — Шпаргалка и Техническое задание (ТЗ)

Документ из двух частей: (1) памятка по подпроектам, (2) техническое задание на систему рассылки SMS.

ЧАСТЬ 1. ПАМЯТКА ПО ПОДПРОЕКТАМ

ClinicHub.Core общая библиотека

Роль	Общие модели и интерфейсы, подключается всеми проектами
Класть сюда	`SmsRequest`, `Patient`, `User`, DTO, интерфейсы сервисов

ClinicHub.Api backend

Роль	ASP.NET Core Minimal API. Хранит заявки, планирует рассылку, управляет очередями и остатками
Адреса	`https://localhost:7180` / `http://localhost:5112`
Документация	`https://localhost:7180/openapi/v1.json` (именно /v1.json)

ClinicHub.Admin Windows (ПК)

Роль	Интерфейс администратора: создание/одобрение заявок, ввод остатков, управление телефонами
Запуск	В списке над ▶ выбери **Windows Machine** → F5

ClinicHub.SmsAgent Android

Роль	Работает на дежурных телефонах: мониторит очередь и отправляет SMS, работает в фоне
Запуск	Android Device Manager → создать/запустить эмулятор → выбрать его → F5
Замечание	Ошибка «System UI isn't responding» → нажми «Wait», эмулятор просто медленный

Запуск всех проектов вместе

- ПКМ по решению → «Configure Startup Projects...» → «Multiple startup projects»
- Для `ClinicHub.Api` и `ClinicHub.Admin` — действие «Start» → OK → F5
- `ClinicHub.SmsAgent` запускается отдельно на телефоне/эмуляторе и подключается к работающему API

Хранение секретов (без попадания в git)

Способ	Команда / пример
User Secrets ✓ рекомендую	`dotnet user-secrets init --project ClinicHub.Api`, затем `dotnet user-secrets set "ApiKey" "значение" --project ClinicHub.Api`
appsettings.Development.json + .gitignore	`"ApiUrl": "https://localhost:7180"` — файл нет в git
Переменные окружения	`$env:CLINICHUB_API_KEY = "значение"` или в `launchSettings.json`
.env + .gitignore	`API_KEY=значение`, `.env` в `.gitignore`

ВАЖНО: текущий код — тестовый

Весь существующий код в проектах — это автоматически сгенерированные шаблоны (Visual Studio), а не рабочая логика. Он нужен только для одного: **проверить, что проект собран правильно** и приложение запускается на каждой платформе.

- **ClinicHub.Api** — шаблонный эндпоинт `/weatherforecast` (случайные температуры) — тестовые данные, удалить при написании ТЗ-функций;
- **ClinicHub.Admin** и **ClinicHub.SmsAgent** — одинаковый «Hello World» со счётчиком кликов — тестовая страница MAUI;
- **ClinicHub.Core** — пустой `Class1.cs`.

Если «Hello World» открывается и на ПК (Windows Machine) и на эмуляторе Android — значит каркас собран верно, и можно удалять шаблоны и писать реальную логику по ТЗ. Выдуманные данные («погода», счётчик, `Class1`) в прод не попадут.

ЧАСТЬ 1. А. ТЕКУЩАЯ СТРУКТУРА ПРОЕКТА

Файлы, которые сейчас есть в решении (на момент составления документа; папки `bin/`, `obj/`, `.vs/` опущены — это временные артефакты сборки).

```
ClinicHub.slnx                  ← файл решения (4 проекта)

ClinicHub.Core/                 ← общая библиотека
├── Class1.cs                   ← ПУСТОЙ шаблон (удалить)
└── ClinicHub.Core.csproj

ClinicHub.Api/                  ← backend (ASP.NET Core Minimal API)
├── Program.cs                  ← точка входа, эндпоинт /weatherforecast (тест)
├── appsettings.json            ← настройки (логирование)
├── appsettings.Development.json← настройки для разработки
├── ClinicHub.Api.http          ← пример запроса REST
└── Properties/launchSettings.json  ← профили запуска (http:5112, https:7180)

ClinicHub.Admin/                ← приложение администратора (MAUI, ПК)
├── MauiProgram.cs              ← точка входа MAUI
├── App.xaml / App.xaml.cs      ← корневой класс приложения
├── AppShell.xaml / .cs         ← навигация (Shell)
├── MainPage.xaml / .cs         ← «Hello World» (тест)
├── Platforms/
│   ├── Android/                ← код Android-платформы
│   ├── iOS/, MacCatalyst/      ← код iOS-платформы
│   └── Windows/                ← код Windows-платформы (App.xaml.cs)
└── Resources/                  ← иконки, шрифты, изображения, стили

ClinicHub.SmsAgent/             ← приложение рассыльщик (MAUI, Android)
├── MauiProgram.cs, App.xaml, AppShell.xaml, MainPage.xaml  ← то же «Hello World» (тест)
├── Platforms/Android/          ← Android-платформа (сюда позже: разрешения SMS)
└── Resources/                  ← ресурсы
```

Обе MAUI-программы (Admin, SmsAgent) пока **идентичны** — это один и тот же шаблон. В дальнейшем они будут различаться: Admin — формой заявок и отчётами, SmsAgent — фоновым сервисом отправки.

ЧАСТЬ 1. Б. ЗАВИСИМОСТИ И ВЕРСИИ — КАК ВОСПРОИЗВЕСТИ СБОРКУ

Проверено на машине разработчика 11.09.2026 (`dotnet --version`, `dotnet workload list`, содержимое csproj).

Б.1 ОС и инструменты

Компонент	Версия	Примечание
ОС	Windows (10/11, 64-бит)	Для сборки Windows/Android проектов. iOS/MacCatalyst собираются только на macOS
Visual Studio	**2026** (VS 18.10, build 18.10.12201.205)	Маст be с workload'ами .NET MAUI + ASP.NET
Java JDK	**21.0.8** (Android Studio OpenJDK)	Путь: `C:\Program Files\Android\openjdk\jdk-21.0.8` — требуется для сборки Android
Android Studio	установлена	Поставляет JDK и инструменты; для эмулятора необязательна, можно через VS

Б.2 .NET SDK

Параметр	Значение
SDK (основной)	`.NET SDK 10.0.401`
Дополнительно установлен	`6.0.428` (не обязателен, остался от других проектов)
Команда проверки	`dotnet --list-sdks`

Б.3 Workload'ы .NET MAUI (dotnet workload list)

Workload	Версия манифеста	Источник
android	36.1.69/10.0.100	VS 18.10.12201.205
ios	26.5.10301/10.0.100	VS 18.10.12201.205
maccatalyst	26.5.10301/10.0.100	VS 18.10.12201.205
maui-windows	10.0.20/10.0.100	VS 18.10.12201.205

Версия пакетов манифеста workload: `10.0.400-manifests.b0700452`. Проверка/установка: `dotnet workload list` / `dotnet workload install maui-android maui-windows`. Установка через Visual Studio Installer → компонент «.NET MAUI» предпочтительнее.

Б.4 Android-инструменты

Компонент	Версия	Путь
Android SDK	API, доступные через VS; проект требует `android` net10	`%LOCALAPPDATA%\Android\Sdk` (у тебя: `C:\Users\xcx7276\AppData\Local\Android\Sdk`)
adb	из Android SDK platform-tools	`\platform-tools\adb.exe`

Эмулятор	создаётся в Android Device Manager (VS)	AVD в `%USERPROFILE%\.android\avd`

Проверка устройств: `& "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe" devices` — если список пуст, эмулятор/телефон не подключён.

Б.5 Целевые платформы (TargetFramework) и версии в csproj

Проект	TFM (TargetFramework(s))	Пакеты NuGet
ClinicHub.Core	`net10.0`	—
ClinicHub.Api	`net10.0`	`Microsoft.AspNetCore.OpenApi 10.0.12`
ClinicHub.Admin	`net10.0-android`; + `net10.0-ios`, `net10.0-maccatalyst` (не на Linux); + `net10.0-windows10.0.19041.0` (на Windows)	`Microsoft.Maui.Controls` версии `$(MauiVersion)` (MauiVersion задаёт MAUI-workload); `Microsoft.Extensions.Logging.Debug 10.0.0`
ClinicHub.SmsAgent	аналогично Admin	аналогично Admin

Прочее в csproj Admin/SmsAgent: `UseMaui=true`, `MauiXamlInflator=SourceGen`, `WindowsPackageType=None`, ApplicationId `com.companyname.clinichub.admin` / `…smsagent`, мин. версии ОС: Android 21, iOS 15, Windows 10.0.17763.

Б.6 Профили запуска и порты (launchSettings.json)

Проект	Профиль	URL	Environment
CLinicHub.Api	`http`	`http://localhost:5112`	Development
ClinicHub.Api	`https`	`https://localhost:7180;http://localhost:5112`	Development
Admin / SmsAgent	MAUI	—	выбор платформы в панели ▶

Б.7 Быстрая проверка «собралось ли» (консоль)

```
dotnet restore ClinicHub.slnx
dotnet build ClinicHub.slnx -c Debug        # сборка всех 4 проектов
dotnet build ClinicHub.Api/ClinicHub.Api.csproj   # только API (быстрый тест)
```

Полная сборка Android из консоли (пример): `dotnet build ClinicHub.Admin -f net10.0-android -c Debug`.

ЧАСТЬ 2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ «АВТОМАТИЗИРОВАННАЯ СИСТЕМА РАССЫЛКИ SMS-УВЕДОМЛЕНИЙ КЛИНИКИ»

Содержание ТЗ:

1. Назначение и цели системы
2. Архитектура и участники
3. Функциональные требования (по блокам)
4. Подсистема планирования и «защита от блокировки» SIM
5. Подсистема остатков и расчёта стоимости SMS
6. Фоновая работа SmsAgent на Android
7. Сущности базы данных (MS SQL)
8. Состав API (ручки и методы)
9. Интеграция с сайтом клиники
10. Нефункциональные требования

1. Назначение и цели

Система обеспечивает **приём заявок на отправку SMS** (как персональных — запись на приём, так и массовых — рассылка многим пациентам) и их **автоматическую, безопасную отправку** через дежурные телефоны с сим-картами, с учётом лимитов операторов и остатков SMS на каждой сим-карте.

Основные цели:

- исключить **ручной ввод SMS по одному** — всё от пользователя идёт через одну форму;
- не допустить **блокировки сим-карт** операторами из-за резких массовых отправок — сервер сам распределяет отправку по интервалам;
- вести учёт **остатков SMS по каждой SIM** и не допускать отправки «в минус»;
- отправлять SMS, когда телефон **заблокирован/в фоне**.

2. Архитектура и участники

Участник	Платформа	Роль в системе
ClinicHub.Admin	Windows	Интерфейс администратора: создание заявок всех типов, одобрение, кнопка «Отправить сейчас», управление телефонами и сим-картами, ввод остатков
ClinicHub.Api	Сервер	Центральный хаб: приём заявок, планирование, распределение по телефонам, учёт остатков, журнал отправок, точки интеграции для сайта клиники
ClinicHub.SmsAgent	Android	Дежурное приложение на 4+ телефонах: постоянно «слушает» сервер, забирает задачи и физически отправляет SMS; работает в фоне на заблокированном телефоне
ClinicHub.Core	—	Общие модели и DTO, используемые всеми участниками
База данных	MS SQL Server	Хранение заявок, очередей, телефонов, сим-карт, остатков, журналов

Число дежурных телефонов **настраиваемо** (по умолчанию 4, может быть больше или меньше).

3. Функциональные требования

3.1 Типы заявок (Request)

Тип	Описание	Пример
Персональная	SMS одному конкретному пациенту/клиенту, подтверждение записи. Одна заявка → одно SMS (может быть из нескольких сегментов)	«Уважаемый Иван, ваша запись к терапевту 12.09 в 14:00»
Массовая	Одна заявка → SMS **многим абонентам**. Списком получателей или по выборке из справочника	«Вакцинация от гриппа доступна с 20.09, запись по телефону…»

Статусы заявки: `Created` (создана) → `Approved` (одобрена) → `Scheduled` (в плане) → `Sending` (идёт отправка) → `Sent` (отправлена) / `Partial` (частично) / `Failed` (ошибка) / `Rejected` (отклонена).

3.2 Планирование отправки REQ-SCHED

Для каждой одобренной заявки задаётся один из режимов отправки:

Режим	Что делает система
Сейчас кнопка «Отправить сейчас»	Нажата после одобрения → заявка сразу попадает в очередь и уходит ближайшей партией по правилам интервалов
За день	Отправка планируется за сутки (например, за 24 часа до времени приёма/события)
За час	Отправка планируется за 1 час до события
За 2 недели	Отправка планируется за 14 дней до события (напоминание/анонс)
Произвольная дата/время	Администратор указывает конкретный момент отправки

Важно: расписание (за день / за час / за 2 недели) означает, что система **сама**, в рассчитанный момент, превращает заявку в очередь отправки — без повторных действий оператора. Кнопка «Сейчас» лишь немедленно запускает уже одобренную заявку.

3.3 Подсистема планирования и «защита от блокировки» SIM REQ-RATE

Сервер (**не телефоны**) решает, когда и с какой номер/телефона уйдёт каждое SMS.

- Для каждого режима отправки задаётся **окно отправки** (например, «в течение дня, с 9:00 до 20:00»), внутри которого сервер разносит SMS.
- Для каждой SIM задаются лимиты: **максимум SMS в минуту**, **в час**, **в сутки** — пороги настраиваются (зависят от оператора).
- Сервер строит очередь SMS и **растягивает её по интервалам**: если заявка на 500 SMS, а лимит 60/час — отправка займёт ~8-9 часов с паузами.
- Очередь распределяется между **всеми доступными телефонами**, учитывая их лимиты и остатки.
- Если ни одна SIM не может отправить сейчас (лимит/остаток исчерпан) — SMS остаётся в очереди с нужным приоритетом, и система отправляет позже; в журнале появляется пометка причины задержки.
- Сервер **не превышает лимиты даже при скоплении очереди** — приоритет безопасности сим-карты выше скорости рассылки.

Настройки лимитов и окон хранятся в БД и могут различаться для разных SIM (разные операторы).

3.4 Управление телефонами и SIM REQ-PHONES

- Система ведёт реестр дежурных телефонов: имя, статус (онлайн/офлайн), зарегистрированное приложение SmsAgent.
- Телефон считается **онлайн**, если SmsAgent периодически (непрерывно, интервал ~10-30 с) подтверждает серверу своё присутствие.
- Для каждого телефона ведётся список сим-карт: номер, оператор, лимиты, остаток SMS, статус.
- Если телефон офлайн — его SIM исключаются из распределения; нагрузку берут остальные.
- Администратор может временно **заблокировать телефон или SIM** (например, SIM «улетела в бан»), и сервер перестанет назначать на неё отправки.

3.5 Остатки и расчёт SMS REQ-BALANCE

Учёт длины SMS: одно «сообщение» = одно реальное SMS по тарифу оператора. Длина формирует число сегментов:

Кодировка	1 SMS	Составное (многоблочное), каждый блок
GSM-7 (латиница/стандарт)	до 160 символов	до 153 символов на блок
Юникод (кириллица и пр.)	до 70 символов	до 67 символов на блок

Расчёт сервером: для каждого текста сервер считает число SMS-сегментов (с учётом кириллицы/юникода) и **списывает именно их** с остатка SIM перед назначением на отправку.

- Ввод/коррекция остатков производится администратором в Admin (вручную, т.к. операторы не дают API) — поле «Текущий остаток», истории изменений.
- После фактической отправки SmsAgent сообщает результат; сервер **списывает** фактическое число сегментов.
- Если остатка не хватает — часть сообщений не назначается и ждёт пополнения; в журнале фиксируется причина.
- На каждую SIM ведётся **история расхода** (дата, заявка, сегментов, остаток после).

Пример: массовая заявка, 120 получателей, текст 150 симв. на кириллице → 150/67 = 3 сегмента на абонента → всего 360 сегментов. Система проверит остатки всех SIM и распределит 360 сегментов по телефонам в пределах лимитов/остатков.

3.6 Фоновая работа SmsAgent на Android REQ-BG

- Приложение работает как **Android Foreground Service** (постоянное уведомление «SmsAgent активен») — система Android не «усыпляет» его.
- Работает при **заблокированном экране** и когда свернуто в фон.
- Циклически (интервал настраивается, ~10-30 с) **опрашивает сервер**: «есть ли задачи для телефона X?»
- При получении задачи: проверяет наличие SMS-разрешений → отправляет → сообщает серверу результат (успех/сегменты/ошибка).
- При сбоях (нет сети, нет разрешений, телефон офлайн) — **повторные попытки** с нарастающей паузой, все факты пишутся на сервер.
- Освобождение: если у телефона несколько SIM — номер SIM выбирается сервером заранее и передаётся в задаче, SmsAgent отправляет с указанной SIM (через настройки/выбор слота, где возможно).
- Приложение отображает статус: зарегистрировано, «слушает», сколько отправил сегодня, остатки SIM (данные с сервера).

Требуемые Android-разрешения: `SEND_SMS` (с Android 11 — `READ_PHONE_STATE` /выбор SIM по возможности), `INTERNET` , `POST_NOTIFICATIONS` , работать поверх блокировки (`FOREGROUND_SERVICE`). Выдача/настройка разрешений документируется для оператора.

7. Сущности базы данных (MS SQL)

Таблица	Ключевые поля
Users	Id, Login, PasswordHash, Role (Admin), IsActive
Patients (справочник получателей)	Id, FullName, Phone, Email, IsSubscribed, Notes
SmsRequests	Id, Type (Personal/Mass), Title, Status, CreatedBy, CreatedAt, PlannedSendAt, SendMode (Now/Day/Hour/2Weeks/Custom), ApprovedBy, ApprovedAt, TextTemplate
RequestRecipients	Id, RequestId, PatientId, Phone, MessageText, SmsSegments, SendAfter, Status, SentAt
Phones	Id, Name, DeviceId (уникальный ключ SmsAgent), IsActive, LastSeenAt, ApiKey
SimCards	Id, PhoneId, Number, Operator, LimitPerMin, LimitPerHour, LimitPerDay, Balance (остаток), IsActive
BalanceLog	Id, SimCardId, Change, Reason, RequestId?, UserId?, CreatedAt, BalanceAfter
SmsQueue (очередь)	Id, RecipientId, SimCardId?, PhoneId?, Priority, PlannedAt, Status, Attempts, LastError
SmsLog (журнал отправок)	Id, QueueId, SimCardId, PhoneId, Segments, Status, SentAt, ResultMessage
RateConfig (общие лимиты)	Id, Name, SendWindowFrom, SendWindowTo, DefaultsPerMin/PerHour/PerDay

Связи: `SmsQueue → SimCards → Phones`, `SmsQueue → RequestRecipients → SmsRequests`. Все изменения в очереди и квотах — транзакционные (MSSQL), чтобы не допустить двойной отправки при конкурентном опросе телефонов.

8. Состав API (ручки и методы)

Группа	Методы
Аутентификация	`POST /api/auth/login`, `POST /api/auth/refresh`
Заявки (Admin)	`GET /api/requests`, `POST /api/requests` (создать), `GET /api/requests/{id}`, `POST /api/requests/{id}/approve`, `POST /api/requests/{id}/reject`, `POST /api/requests/{id}/send-now` (кнопка «Отправить сейчас»), `GET /api/requests/{id}/status`
Получатели	`GET /api/patients`, `POST /api/patients`, `PUT /api/patients/{id}`
Телефоны и SIM	`GET /api/phones`, `POST /api/phones`, `POST /api/phones/{id}/sim`, `PUT /api/sim/{id}/balance` (ввод остатка), `POST /api/sim/{id}/activate|deactivate`, `GET /api/sim/{id}/balance-log`
Выдача задач SmsAgent	`POST /api/agent/register` (регистрация телефона), `GET /api/agent/{deviceId}/tasks` (получить задачи — только свои), `POST /api/agent/{deviceId}/tasks/{taskId}/result` (результат отправки), `POST /api/agent/{deviceId}/heartbeat` (подтверждение «жив»)
Интеграция с сайтом клиники	`POST /api/webhook/requests` (сайт создаёт заявку на запись/напоминание), `GET /api/webhook/requests/{id}/status` (сайт узнаёт статус), `POST /api/webhook/requests/{id}/callback` (сервер сообщает сайту итог отправки)

REST + JSON, документация через OpenAPI (`/openapi/v1.json`). Все ручки, кроме `login`, защищены токеном (Bearer JWT); связка Agent использует отдельные `ApiKey` телефона.

9. Интеграция с сайтом клиники

- Весь обмен — по REST/JSON через открытые ручки **/api/webhook/...**.
- Сайт может **создавать заявки** (запись к врачу, перенос, напоминания за время) — без участия оператора кладет их в ту же систему статусов и планирования.
- Сайт может **узнавать статусы** своих заявок и получать уведомление о доставке/ошибке через callback.
- Ключ интеграции (клиника ↔ Api) настраивается отдельно, чтобы сайт не имел доступа к административным ручкам.
- Зарезервированы поля `ExternalId` (id заявки на сайте) — для сопоставления заявок между сайтом и системой.

10. Нефункциональные требования

- **БД:** MS SQL Server (Express 2019+ достаточно для объёма клиники); доступ через `appsettings.json` (строка подключения — секрет, в User Secrets в dev).
- **Надёжность:** очереди и списание остатков — атомарные операции; повторная отправка исключена даже при падении соединения (идемпотентность задач по `taskId`).
- **Производительность:** поддержка массовой рассылки на тысячи получателей — очередь в БД, обработка по интервалам; UI Admin не блокируется созданием большой заявки.
- **Безопасность:** JWT для Admin, ApiKey для телефонов, интеграционный ключ для сайта; журналирование действий администратора.
- **Мониторинг:** дашборд в Admin: сколько в очереди, статусы телефонов, остатки SIM, скорость отправки сегодня, ошибки.
- **Конфигурируемость:** число телефонов, лимиты, окна отправки, кодировка текста — без пересборки, параметрами в БД.
